

Giorno 12 — REST avanzato e introduzione alla sicurezza

Esercizio 1 — Path variables, exception handling e service layer

Raffinerai la tua API REST introducendo il Service Layer per separare la logica di business dal controller, gestirai le eccezioni in modo centralizzato e utilizzerai path variables e query params.

Cosa fare:

- Refactoring: estrai la logica CRUD dal controller e crea un `@Service` `StudenteService`. Il controller deve delegare TUTTE le operazioni al service. Il service usa il repository.
- Crea un'eccezione personalizzata `StudenteNotFoundException` (che estende `RuntimeException`). Lanciala nel service quando un'operazione cerca uno studente con ID non esistente.
- Crea un `@RestControllerAdvice` `GlobalExceptionHandler` con un metodo annotato `@ExceptionHandler(StudenteNotFoundException.class)` che restituisca una risposta JSON strutturata: `{"errore": "Studente non trovato", "id": 99, "timestamp": "..."} con codice HTTP 404.`
- Aggiungi un endpoint `GET /api/studenti/cerca` con un parametro query `@RequestParam` `String cognome` che restituisce tutti gli studenti con quel cognome. Se nessuno trovato, restituisce lista vuota (non 404).
- Aggiungi un endpoint `GET /api/studenti/{id}/corso` che restituisce solo il corso di laurea dello studente con quell'ID, come stringa JSON. Se lo studente non esiste, usa l'eccezione del punto 2.
- Testa la gestione degli errori: chiama `/api/studenti/9999` e verifica che la risposta abbia codice 404 e il corpo JSON corretto con timestamp. Usa Postman per verificare gli header della risposta.

Consegna: L'architettura finale deve rispettare la separazione: Controller -> Service -> Repository.

Esercizio 2 — Prima configurazione della sicurezza REST

Aggiungi Spring Security al progetto e configuralo per proteggere gli endpoint dell'API. Implementa l'autenticazione HTTP Basic e una prima distinzione tra endpoint pubblici e protetti.

Cosa fare:

- Aggiungi al `pom.xml` la dipendenza `spring-boot-starter-security`. Riavvia l'applicazione: cosa succede se provi ad accedere agli endpoint?
- Crea una classe `SecurityConfig` annotata con `@Configuration` e `@EnableWebSecurity`. Crea un bean `SecurityFilterChain` che configuri: `GET /api/studenti` accessibile senza autenticazione, tutti gli altri endpoint richiedono autenticazione.
- Configura un utente in-memory nel `SecurityConfig` con `username='admin'`, `password='admin123'` e ruolo `ADMIN`.
- Testa con Postman: la `GET /api/studenti` deve funzionare senza credenziali. La `POST /api/studenti` deve richiedere Basic Auth con `username/password`. Cosa risponde il server se inserisci credenziali sbagliate? (Nota il codice HTTP)
- Aggiungi un secondo utente con `username='user'` e ruolo `USER`. Configura la security in modo che: gli utenti con ruolo `USER` possano solo fare `GET`, solo gli `ADMIN` possano fare `POST`, `PUT` e `DELETE`.
- NOTA: Disabilita la protezione CSRF per le API REST (`csrf().disable()`) nel `SecurityConfig`

Consegna: Crea una tabella in formato testo (nel `README.md`) che mostri per ogni endpoint la URL, il metodo HTTP, e chi può accedervi (tutti / USER / ADMIN).